PocketSPAR: On-Device Visual Planning with Aligned Representations and Learned Heuristic Search

Misagh Soltani[1], **Forest Agostinelli**[1],

[1]Department of Computer Science, University of South Carolina, Columbia, SC 29208, USA
msoltani@email.sc.edu, foresta@cse.sc.edu

Abstract

The deployment of automated planning algorithms that operate on pixel-based observations in real-world settings is often hindered by the challenge of visual distribution shift: when the observation at test time differs from the training distribution due to lighting. In this work, we demonstrate PocketSPAR, an iOS application that runs Stable Planning through Aligned Representations (SPAR) in Model-Based Reinforcement Learning on device locally and can find solutions to complex planning problems. SPAR can solve long-horizon planning problems in environments with noisy real-world observations while being trained entirely in simulation, by learning an alignment network that maps noisy observations to the clean discrete latent state.

Demo Video — https://github.com/misaghsoltani/PocketSPAR_Demo/raw/refs/heads/main/submission_bundle/video/pocketspar_icaps26_demo.mp4

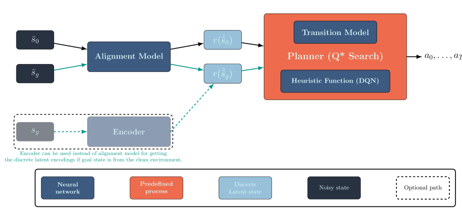

Figure 1: SPAR planning pipeline. A noisy observation is aligned to the clean discrete latent state and then using the fixed world model and heuristic, Q* search can be executed.

Introduction and Background

Planning algorithms that operate on pixel-based observations have the potential to enable a wide range of real-world applications. In recent years, there have been advances in learning-based planning algorithms that can operate on visual observations (Agostinelli et al. 2019). In environments where data collection is expensive, such as real-world robotics, model-based reinforcement learning (MBRL) algorithms are used to learn a world model, and use it for training a heurristic function that can be used for search. However, in domains with long-horizon planning problems, the learned model can accumulate small errors over long rollouts, leading to poor prediction of the future states. Moreover, the ability to re-identify states during search is crucial in search algorihms. When world-models are used, small prediction errors can lead to incorrect state re-identification, which can cause search algorithms to fail. DeepCubeAI (Agostinelli and Soltani 2024) addresses this challenge by learning a discrete latent representation of the state space, which allows for robust state re-identification during search, and corrects for small prediction errors through rounding. However, DeepCubeAI assumes that the clean observations generated from the world model are the same as the observations at test time, which is not the case in real-world applications due to visual distribution shift. SPAR (Soltani and Agostinelli 2025) extends DeepCubeAI to address this challenge by learning an alignment network that maps noisy observations to the same clean discrete latent state that the world model and heuristic were trained on. This allows the planner to operate reliably even when the observation distribution at test time differs from the training distribution, by only training an alignment network to adapt to the new observation distribution while keeping the world model and heuristic fixed. Figure 1 shows the SPAR planning pipeline, and Figure 2 shows the adaptation mechanism that retrains only the alignment network when the observation distribution changes.

SPAR's adaptation mechanism is particularly important for real-world applications, where the observation distribution can change due to lighting variation, camera noise, or other factors. By separating the perceptual mapping from the planning, SPAR allows for training a single world model and heuristic that can be reused across different observation distributions, and by training on noisy obsevations in simulation, SPAR can learn to align to real observations without ever seeing them during training. Figure 2 highlights this decoupling by showing that the clean encoder pathway remains fixed while only the alignment pathway is updated for shifted observations. This makes SPAR a promising approach for deploying planning algorithms that operate on pixel-based observations in real-world settings.

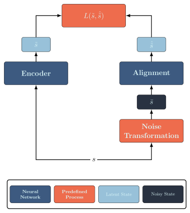

Figure 2: SPAR adaptation mechanism. When the observation distribution changes, only the alignment network is retrained to match the clean encoder latent state.

On-Device Visual Planning with PocketSPAR

We introduce PocketSPAR, an iOS application that runs SPAR on device locally and uses the device's camera to capture visual observations, uses an alignment network that is trained on noisy synthetic observations to map them to the clean discrete latent state, and executes Q* search (Agostinelli et al. 2024) with the fixed world model and heuristic to find a solution. We demonstrate the application in the $3 \times 3 \times 3$ Rubik's Cube domain, where the planner finds a sequence of moves to solve a scrambled cube from a given start state to a given goal state. The application consists of two main components: SPARBundleCreator, a macOS application that converts a trained SPAR checkpoint into a self-contained mobile deployment bundle, and PocketSPAR, the iOS runtime that consumes the prepared bundle and provides an interface for guided two-view camera capture, on-device reconstruction preview, and Q* search execution with live progress reporting. Figure 3 shows the bundle preparation workspace, and Figure 4 shows the on-device capture interface used before reconstruction and search.

System Components

SPARBundleCreator, shown in Figure 3, is a macOS application that converts a trained SPAR checkpoint into a self-contained mobile deployment bundle. It exports the alignment network, decoder, transition model, and heuristic from PyTorch to Core ML format, packages them with a SearchManifest specifying the observation contract (2 views at 32×32 pixels, 12-action space, 400-dimensional latent state) and default search parameters, and produces an export-parity report.

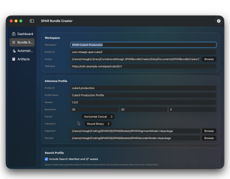

Figure 3: SPARBundleCreator workspace for packaging a trained SPAR checkpoint into a mobile deployment bundle with model assets and search metadata.

PocketSPAR, shown in Figure 4, is the iOS runtime that consumes the prepared bundle. It provides guided two-view camera capture alongside photo and file import, an on-device reconstruction preview that displays the inferred start and goal states before search is initiated, a Q* search workspace with live progress reporting including iteration count and node statistics, and three configurable runtime profiles (Safe Streaming, Balanced, and Uncapped) that trade memory footprint against throughput.

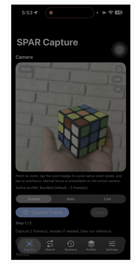

Figure 4: PocketSPAR guided capture interface for collecting the two visual observations of Rubik's Cube, used to reconstruct the start or goal state on device.

Robustness. The alignment network maps visual distribution shift without modifying the world model or heuristic. The reconstruction preview shows the reconstructed state to the user, enabling recapture when alignment accuracy is low.

Generalization. The world model and goal-conditioned heuristic are fixed after training. Adapting to a new observation distribution requires retraining only the alignment network. Adapting to a new domain requires retraining the alignment network for the new observation distribution while leaving the world model and heuristic intact. Deploying to a new device requires running SPARBundleCreator on the updated checkpoint, which handles conversion, validation, and packaging.

Scalability. Q* search depth and node budget are configurable at runtime through the Settings panel. The three runtime profiles govern the trade-off between memory footprint and throughput on the target device. Solution latency in the Rubik's Cube domain falls within interactive bounds on current hardware using the Neural Engine through Core ML.

References

Agostinelli, F.; McAleer, S.; Shmakov, A.; and Baldi, P. 2019. Solving the Rubik's cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1(8): 356–363.

Agostinelli, F.; Shperberg, S. S.; Shmakov, A.; McAleer, S.; Fox, R.; and Baldi, P. 2024. Q* search: Heuristic search with deep q-networks.

Agostinelli, F.; and Soltani, M. 2024. Learning Discrete World Models for Heuristic Search. *Reinforcement Learning Journal*, 4: 1781–1792.

Soltani, M.; and Agostinelli, F. 2025. Stable Planning through Aligned Representations in Model-Based Reinforcement Learning. In *NeurIPS 2025 Workshop on Embodied World Models for Decision Making*.